Foundations of Deep Learning—Final Exam

Master's Degree in AI & Society (PSL-PSAI, Spring 2026)

Wednesday 13th May 2026, 1h30

Name:

Signature:

Instructions.

Closed book. No electronic devices. Read each question carefully before answering. Do not detach any pages. **Prioritise quality over quantity.**

A sheet of useful formulas and blank answer sheets are provided at the end of the exam.

Section	Topic	Suggested Time	Points
1	True / False	15 min	16
2	Multiple Choice	20 min	24
3	Short Answer	25 min	30
4	Exercises	30 min	30
Total		**90 min**	**100**

Do not turn the page until instructed to do so.
Good luck!

Part 1—True / False (16 points)

Instructions: Cross **True** or **False**. No explanation is required.

Scoring: +2 correct, -1 incorrect, 0 blank.

1.1 A neural network with no hidden layers (input directly connected to output) can only learn linear decision boundaries.

◯ True ◯ False

1.2 The universal approximation theorem states that a neural network with a sufficient number of hidden layers can approximate any continuous function on a compact domain to arbitrary precision, provided the activation functions are non-linear.

◯ True ◯ False

1.3 Consider the mean squared error (MSE) and sum of squared errors (SSE) loss functions:

$$\mathcal{L}^{\text{SSE}}(\mathbf{w}) = \sum_{n=1}^{N}(y_n - t_n)^2, \qquad \mathcal{L}^{\text{MSE}}(\mathbf{w}) = \frac{1}{N}\sum_{n=1}^{N}(y_n - t_n)^2$$

where $y_n = y(\mathbf{x}_n, \mathbf{w})$ is the model output and t_n is the target. A gradient descent update using $\mathcal{L}^{\text{MSE}}$ with learning rate η is equivalent to a gradient descent update using $\mathcal{L}^{\text{SSE}}$ with learning rate η/N.

◯ True ◯ False

1.4 Backpropagation requires the use of canonical output/loss pairs (e.g. softmax + cross-entropy for classification, linear output + SSE for regression).

◯ True ◯ False

1.5 Convolutional neural networks are equivariant to rotations of the input image: rotating the input by any angle produces a correspondingly rotated set of feature maps.

◯ True ◯ False

1.6 A convolutional layer with C_{out} filters of kernel size $H \times W$—equal to the full spatial dimensions of the input feature map of shape $H \times W \times C_{in}$—with no padding and stride 1, is equivalent to a fully connected layer mapping $H \times W \times C_{in}$ inputs to C_{out} outputs.

◯ True ◯ False

1.7 The position-wise MLP in a transformer layer is applied independently to each token and does not allow information exchange between positions. Context across positions is handled entirely by the self-attention sub-block.

◯ True ◯ False

1.8 AI systems trained on images can recognise objects in photographs with high accuracy. This demonstrates that they have acquired the concept of *objectness*—the capacity to perceive the world as composed of discrete, persistent entities.

◯ True ◯ False

Part 2—Multiple Choice (24 points)

Instructions: Each question states the number of correct answers k. You **must tick exactly k boxes**. No explanation is required.

Scoring: 4 points if all k ticked options are correct. 2 points if exactly $k-1$ ticked options are correct. 0 otherwise. Furthermore, any answer with a number of ticked boxes different from k scores 0.

2.1 The table below summarises the canonical output/loss pairs for different prediction tasks. Which of the following statements are correct? (**3 correct answers**)

Task	Output activation	Loss
Regression	Identity	SSE
Binary classification	Sigmoid	BCE
Multi-class classification	Softmax	Cross-entropy

☐ (A) In all three canonical pairs, the gradient of the loss with respect to the pre-activation of the output layer simplifies to $\mathbf{y} - \mathbf{t}$, where $\mathbf{y}$ is the predicted output and $\mathbf{t}$ is the target.

☐ (B) The three canonical pairs all arise from maximum likelihood estimation under different assumptions about the noise distribution of the targets.

☐ (C) For a multi-label classification problem—where multiple classes can be simultaneously active—the softmax + cross-entropy pair is the appropriate choice.

☐ (D) The identity output activation used in regression is appropriate because it allows the output to take any real value, matching the unbounded range of continuous targets.

☐ (E) Minimising the cross-entropy loss is strictly equivalent to maximising classification accuracy: both objectives are minimised by the same set of parameters.

2.2 Which of the following statements about optimisation and the geometry of loss landscapes are correct? (**3 correct answers**)

☐ (A) In high-dimensional parameter spaces, most critical points of the loss are local minima, since the probability that a critical point is a saddle point decreases as the number of parameters grows.

☐ (B) In neural networks with many parameters, local minima tend to have loss values much higher than the global minimum, making gradient descent likely to get trapped in poor solutions.

☐ (C) The stochasticity introduced by mini-batch SGD helps escape saddle points by providing gradient noise that can push the optimiser away from flat regions where a deterministic gradient would stall.

☐ (D) Gradient descent applied to a loss surface with both stiff (high curvature) and soft (low curvature) directions will oscillate in the stiff directions if the learning rate is large enough to make progress in the soft directions.

☐ (E) At a critical point of the loss, the Hessian matrix H characterises the local geometry: a minimum requires all eigenvalues of H to be positive, a maximum requires all to be negative, and a saddle point has eigenvalues of both signs.

2.3 Consider L2 regularisation (weight decay) applied to a neural network:

$$\tilde{\mathcal{L}}(\mathbf{w}) = \mathcal{L}(\mathbf{w}) + \lambda \|\mathbf{w}\|^2$$

Which of the following statements are correct? (**2 correct answers**)

☐ (A) L2 regularisation has a probabilistic interpretation: minimising the regularised loss $\tilde{\mathcal{L}}(\mathbf{w})$ is equivalent to maximum likelihood estimation under the prior assumption that the weights are drawn from a Gaussian distribution $p(\mathbf{w}) \propto e^{-\lambda \|\mathbf{w}\|^2}$.

☐ (B) The gradient of the regularised loss adds a term $2\lambda\mathbf{w}$ to the standard gradient, which decays the weights towards zero at every gradient step, independently of the data.

☐ (C) Geometrically, L2 regularisation selectively squeezes the soft directions of the loss landscape—those with low curvature—leaving the already stiff directions unchanged, which makes the contours more uniform.

☐ (D) Increasing λ always improves generalisation performance by reducing the model's tendency to overfit the training data.

☐ (E) L2 regularisation reduces the effective number of parameters in the model—in the limit $\lambda \to \infty$, the model is forced to use all its parameters to their maximum capacity to minimise the loss.

2.4 Which of the following statements about image segmentation are correct? (**3 correct answers**)

☐ (A) In semantic segmentation, the output of the network is a tensor of shape $H \times W$, where each spatial position (h, w) contains the predicted class index for that pixel.

☐ (B) The training loss for semantic segmentation is the sum of cross-entropy losses computed independently at each pixel: $\mathcal{L} = \sum_{h,w} \mathcal{L}^{CE}(h, w)$.

☐ (C) In a U-Net architecture, skip connections between the encoder and decoder allow the network to combine high-resolution spatial information from early layers with the semantic features computed at the bottleneck.

☐ (D) The encoder in a segmentation network progressively decreases the spatial resolution of the feature maps while extracting increasingly abstract semantic features, and the decoder recovers spatial resolution through upsampling operations such as transpose convolution.

☐ (E) The decoder in a segmentation network can be implemented using transpose convolutions to upsample feature maps. Transpose convolutions learn to reverse the exact spatial transformation applied by the corresponding encoder convolutions, recovering the original input to each encoder layer.

2.5 Which of the following statements about pre-transformer approaches to sequence modelling are correct? (**2 correct answers**)

☐ (A) An autoregressive language model generates text by sampling all output tokens simultaneously, conditioned on the input sequence.

☐ (B) n-gram language models approximate the probability of a sequence by assuming each token depends only on the previous L tokens. However, they are infeasible for large context sizes.

☐ (C) RNNs share parameters across all time steps, which allows them to process sequences of any length with a fixed number of parameters.

☐ (D) RNNs address the sequential processing limitation of n-gram models: since each hidden state depends only on the current input, all time steps can be computed in parallel.

☐ (E) The bottleneck problem in RNNs refers to the vanishing gradient: as the sequence grows longer, gradients must flow through more time steps and become too small to train the early layers of the network.

2.6 Which of the following statements about positional encodings in transformer models are correct? (**3 correct answers**)

☐ (A) Positional encodings are necessary because the self-attention mechanism is permutation-equivariant: without positional information, the model assigns the same representation to a token regardless of its position in the sequence.

☐ (B) Sinusoidal positional encodings generalise naturally to sequences longer than those seen during training, since they are defined by a fixed mathematical formula for any position n, without requiring any learned parameters.

☐ (C) Positional encodings are applied after the transformer layers, allowing the model to first build contextualised representations and then inject position information at the output stage.

☐ (D) A transformer encoder trained without positional encodings would produce identical output representations for all tokens in a sequence.

☐ (E) Positional encodings must have the same dimension D as the token embeddings, since they are combined with the token representations before being fed into the transformer.

Part 3—Short Answer (30 points)

Instructions: Answer concisely. Use equations if/where appropriate. **Aim for 5–10 sentences per sub-question.** Partial credit is awarded: a well-reasoned incomplete answer scores more than a blank.

3.1 Generalisation in Deep Learning (15 points)

(a) (4 pts) Define the concepts of bias and variance of a model, and connect them to the phenomena of underfitting and overfitting.

(b) (5 pts) In practice, model complexity is controlled by hyperparameters. How do you select the best hyperparameters, and how do you know when a model is overfitting? Describe how generalisation performance evolves as a function of model complexity according to the classical picture.

(c) (6 pts) Modern deep learning sometimes violates the classical prediction. Explain in what sense, identifying the specific point at which this occurs. Sketch generalisation performance as a function of model complexity, labelling the key regimes and the critical point.

3.2 Transfer Learning (15 points)

You have access to a ResNet-50 convolutional neural network pretrained on ImageNet (1.2 million labelled images, 1000 classes). You want to use it as a starting point for two different target tasks:

- **Task A:** Classify skin lesion images as benign or malignant. You have 500 labelled dermatological images.
- **Task B:** Classify satellite images into 10 land-use categories (forest, urban, farmland, ...). You have 80,000 labelled satellite images.

(a) (4 pts) Briefly describe the two main transfer learning strategies discussed in the course. What is the key difference between them in terms of which parameters are updated during training?

(b) (6 pts) For each task, argue which strategy you would recommend and why. Your answer should consider the size of the target dataset, the similarity between source and target domains, and the risk of overfitting.

(c) (5 pts) In both cases, early CNN layers (detecting edges, textures, colour gradients) are more likely to transfer successfully than later layers. Explain why this is the case.

Part 4 — Exercises (30 points)

Instructions: Answer concisely. Show calculations if appropriate. Partial credit is awarded: a well-reasoned incomplete answer scores more than a blank.

4.1 Encoder-Decoder Architecture (15 points)

Consider the following architecture for semantic segmentation of 32×32 colour images into $C = 4$ classes. It follows a simple encoder-decoder scheme where max-pooling and unpooling layers are used to downscale and upscale the spatial dimensions, respectively. All kernels are square.

The table below summarises the layer characteristics and uses the following notation: C_{out} = number of output channels (filters), M = kernel size, S = stride, P = padding.

Layer	C_{out}	M	S	P	Notes
CONV-1	8	3	1	1	
POOL-1	—	2	2	0	Max pooling
CONV-2	16	3	1	1	
POOL-2	—	2	2	0	Max pooling
CONV-BN	32	3	1	1	Bottleneck
UNPOOL-1	—	2	2	—	Unpooling
CONV-3	16	3	1	1	
UNPOOL-2	—	2	2	—	Unpooling
CONV-4	8	3	1	1	
CONV-OUT	?	1	1	0	

The architecture is illustrated in Figure 1.

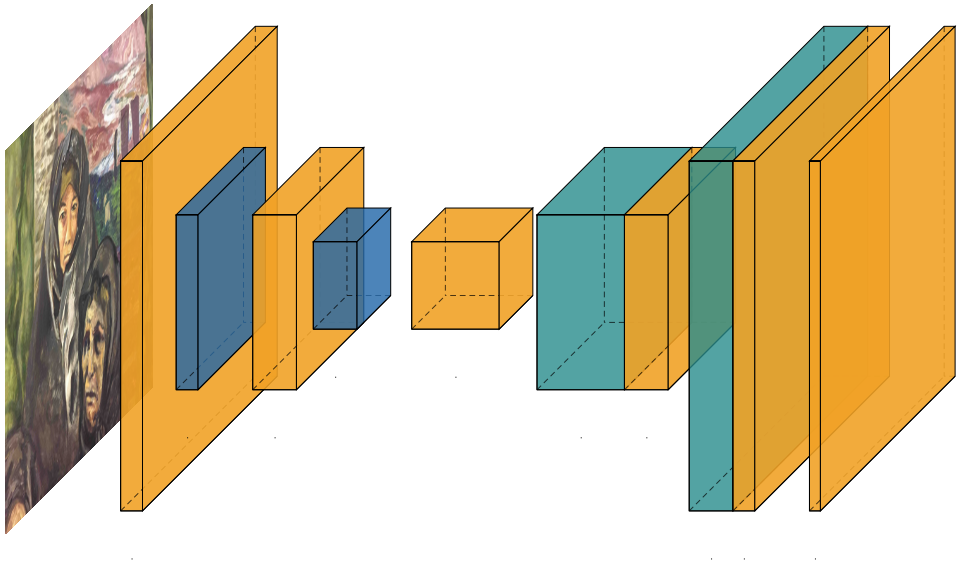

Figure 1: Encoder-decoder architecture for Exercise 4.1. Convolutions (CONV) are in orange, max-pooling (POOL) in blue and unpooling (UNPOOL) in green.

(a) (2 pts) State the output dimensions of CONV-OUT—i.e. the number of filters and the spatial dimensions of the output tensor. Explain why these dimensions are known in advance for a segmentation task, without needing to compute them from the architecture.

(b) (8 pts) Complete the table below. Write output dimensions as (H, W, C) and the total number of parameters (weights+biases) for each layer.

Layer	Output dimensions	# params
CONV-1		
POOL-1		
CONV-2		
POOL-2		
CONV-BN		
UNPOOL-1		
CONV-3		
UNPOOL-2		
CONV-4		
CONV-OUT		

(c) (5 pts) What is the total number of trainable parameters? Which layer(s) contribute the most? Compare the total parameter count of the network to that of a single fully connected layer mapping the input image directly to the output segmentation map—what does this reveal about the efficiency of convolutional layers?

4.2 Automatic differentiation (15 points)

Consider a minimal neural network with scalar input $x \in \mathbb{R}$, learnable parameters $w \in \mathbb{R}$, $b \in \mathbb{R}$, target $t \in \mathbb{R}$, and loss:

$$y = \sigma(wx + b), \qquad \mathcal{L} = (y - t)^2$$

where $\sigma(a)$ is the **sigmoid function**. The computational graph is given below. Each node v_i corresponds to a quantity in the network and $v_9 = \mathcal{L}$.

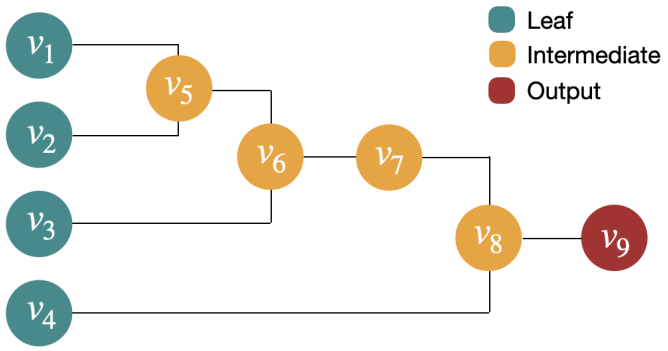

Figure 2: Computational graph for Exercise 4.2

The values for a particular training example are: $x = 2,\ w = 1,\ b = -2,\ t = 1$.

(a) (2 pts) The leaf nodes correspond to $v_1 = x$, $v_2 = w$, $v_3 = b$, $v_4 = t$. For each intermediate and output node $v_5, \ldots, v_9$, write the explicit expression in terms of the nodes it depends on.

(b) (4 pts) Carry out the **forward pass**. Compute the value of each node v_1 through v_9.

(c) (9 pts) Carry out the **backward pass** using the adjoint variable notation $\bar{v}_i = \partial\mathcal{L}/\partial v_i$. Compute:

$$\frac{\partial \mathcal{L}}{\partial y}, \quad \frac{\partial \mathcal{L}}{\partial (wx + b)}, \quad \frac{\partial \mathcal{L}}{\partial w}.$$

By how much is the gradient $\partial\mathcal{L}/\partial(wx + b)$ attenuated with respect to $\partial\mathcal{L}/\partial y$? Is this the best or worst case for the sigmoid? What happens to this attenuation in a deep network with L sigmoid layers, and what does this imply for training?

Extra Pages for Exercises

Extra Pages for Exercises

Useful Formulas

Activation Functions

$$\sigma(a) = \frac{1}{1+e^{-a}}, \qquad \sigma'(a) = \sigma(a)(1-\sigma(a))$$

$$\text{ReLU}(a) = \max(0, a), \qquad \text{ReLU}'(a) = \begin{cases} 1 \text{ if } a > 0 \\ 0 \text{ if } a < 0 \end{cases}$$

$$\text{softmax}(a_i) = \frac{e^{a_i}}{\sum_j e^{a_j}}$$

Adjoint Variables

For a computational graph with nodes $v_1, \ldots, v_n$ and loss $\mathcal{L} = v_n$, the adjoint of node v_i is $\bar{v}_i \equiv \partial \mathcal{L} / \partial v_i$.

Reverse recursion (from output to inputs):

$$\bar{v}_i = \sum_{j \in \text{ch}(i)} \bar{v}_j \frac{\partial v_j}{\partial v_i}$$

where $\text{ch}(i)$ denotes the set of children of node v_i in the graph.

Convolutional Layer

For a convolutional (or pooling) layer with input size H_{in}, kernel size M, padding P, and stride S:

$$H_{out} = \left\lfloor \frac{H_{in} + 2P - M}{S} \right\rfloor + 1$$

Parameter count (weights + biases) for a convolutional layer with C_{in} input channels, C_{out} output channels, and kernel size M:

$$(C_{in} \cdot M^2 + 1) \cdot C_{out}$$